

# Multi-UAV Path Planning for Mobile Edge Computing with High-Density Mobile Devices

Lihan Liu, Hongrui Miao, Chunhui Qu, Zhuwei Wang, Haijun Zhang, *Fellow, IEEE*, Zhidu Li, *Senior Member, IEEE*,

**Abstract**—This paper addresses the challenges of unmanned aerial vehicle (UAV) assisted mobile edge computing (MEC) in high-density user mobility scenarios, a field that has not been extensively explored in current research. We introduce a novel deep reinforcement learning (DRL) framework, named “ $j$ -PPO+EN-ConvNTM”, specifically designed to optimize MEC performance in urban environments with user mobility. The framework integrates spatiotemporal data modeling and spatial transformer network (STN) through an enhance Convolution Neural Turing Machine (EN-ConvNTM) module, which includes a three-dimensional external memory. It also features a joint continuous and discrete action decision-making module, termed joint proximal policy optimization ( $j$ -PPO). This design enables effective handling of the dynamic and complex nature of urban mobility patterns. The proposed approach extends the PPO technique to accommodate joint continuous and discrete action decisions, thereby enhancing UAV adaptability and efficiency in providing MEC services. Extensive simulations demonstrate significant improvements over all baseline models, particularly in terms of equilibrium efficiency and service continuity in high-density scenarios. Our research addresses a critical gap in existing UAV-assisted MEC studies, which primarily focus on static or low-mobility user scenarios, and supports the development of more robust and efficient smart city applications, meeting the real-world demands of modern urban infrastructures.

**Index Terms**—High-density mobile devices, joint continuous and discrete actions, MEC, path planning, UAV.

## I. INTRODUCTION

THE advent of smart cities and the Internet of Things (IoT) has given rise to an unprecedented increase in data generation, particularly in high-density urban areas where a larger number of IoT devices have the task offloading requirements within a specified geographic area. This surge necessitates robust and efficient computing solutions, especially for applications requiring low latency and high bandwidth. MEC has emerged as a promising paradigm by deploying computational

resources closer to the data source, thereby reducing latency and network congestion [1]. However, the dynamic nature of urban environments, especially with mobility user and IoT devices, presents significant challenges in terms of service continuity and quality of experience.

UAVs offer a versatile solution to these challenges. Their mobility and flexibility enable them to deliver on-demand MEC services in various scenarios, especially where user density is high and dynamically changes [2]. UAVs can adaptively adjust their positions to maintain optimal service levels, addressing issues such as signal strength fluctuations and user demand variations [3]. This is particularly relevant in scenarios where ground infrastructure is insufficient or temporarily overloaded, such as during special events or emergencies [4].

Although UAVs provide a great potential, integrating UAVs into MEC architectures faces its own set of challenges. These include ensuring reliable and secure communication, efficient resource allocation, and effective integration with existing network infrastructure [5]. Moreover, the mobility of UAVs introduces additional complexities in terms of trajectory optimization and resource management, which are crucial for sustainable operation [6]. Addressing these challenges is essential to fully harness the potential of UAV-assisted MEC in high-density user environments, paving the way for a new era of smart city applications.

The integration of UAVs with MEC systems has been widely explored, but most existing research mainly focuses on scenarios with static or low-mobility users, inadequately addressing high-density environments. This limitation is critical because user mobility patterns significantly influence the performance of MEC systems. In high-density scenarios, such as urban centers or during large public events, user movements tend to be highly dynamic and unpredictable. This variability in user movement, in turn, results in significant fluctuations in computational resources demands and network connectivity requirements [7]. Current UAV-assisted MEC solutions can not be optimized for these conditions, typically leading to inefficient resource allocation, increased latency, and degraded overall system performance.

Moreover, the majority of current research fails to fully investigate the complexities of UAV trajectory control and resource allocation in high-density mobility environments. Efficient UAV trajectory planning is crucial for ensuring optimal coverage and service quality, especially when dealing with a large number of mobile users [8]. Additionally, maintaining stable and secure communication links in such dynamic environments presents significant challenges, requiring innovative

L. Liu is with the School of System Science and Statistics, Beijing Wuzi University, Beijing 101149, China (e-mail: liulihan@bwu.edu.cn).

H. Miao is with the Min H. Kao Department of Electrical Engineering and Computer Science, University of Tennessee, Knoxville 37996, USA (e-mail: hmiao1@vols.utk.edu).

C. Qu is with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China (e-mail: quch@aircas.ac.cn).

Z. Wang is with the School of Information Science and Technology, Beijing University of Technology, Beijing 100124, China (e-mail: wangzhuwei@bjut.edu.cn).

H. Zhang is with the Beijing Engineering and Technology Research Center for Convergence Networks and Ubiquitous Services, University of Science and Technology Beijing, Beijing 100083, China (e-mail: haijunzhang@ieee.org).

Z. Li is with the School of Communications and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: lizd@cqupt.edu.cn).

approaches to ensure consistent service delivery [9].

This paper aims to address these gaps by focusing specifically on UAV-assisted MEC in high-density user mobility scenarios. The objective is to develop an optimized framework that dynamically adapts to user densities and mobility patterns, ensuring efficient resource utilization and maintaining high-quality service levels. This involves not only improving UAV trajectory planning and resource management, but also enhancing the integration of UAVs into existing MEC infrastructure to meet the unique demands of mobility user environments.

Our work presents a deep learning framework, termed “ $j$ -PPO+EN-ConvNTM”, designed to address the complexities of UAV-assisted MEC in high-density user mobility scenarios. This framework integrates spatiotemporal data analysis with advanced feature extraction via the “EN-ConvNTM” module, and incorporates a sophisticated decision-making process through the  $j$ -PPO module, which handles both continuous and discrete actions. The EN-ConvNTM module features a three-dimensional (3-D) external memory and a STN, enabling it to effectively capture and interpret long-term 3-D observational data [10]. This capability is crucial for accurate decision-making in dynamic environments, and is supported by a novel read-write mechanism specifically adapted to complex urban mobility patterns.

To the best of our knowledge, there is currently scant research on algorithmic solutions for multi-UAV path planning in the context of MEC with high-density mobile devices. This study seeks to fill this research gap, and the main contributions are summarized as follows.

- A multi-UAV-assisted MEC system framework is specifically designed for high-density mobile device scenarios. Detailed analyses are conducted on the device mobility model, channel model, computing model, delay-energy consumption model, and UAV flight model. An optimization problem is then formulated to dynamically control UAV flight trajectories, task offloading strategies, and charging indicators, with the objectives of minimizing energy expenditure, maximizing data collection efficiency, and ensuring fairness.
- A novel algorithm framework integrating EN-ConvNTM and DRL is proposed, meticulously designed for scenarios involving multiple UAVs and high-density mobile users, addressing critical challenges of delays and energy consumption. The proposed algorithm framework achieves autonomous learning and intelligent updates through efficient integration of spatiotemporal information and environmental interactions.
- By deeply fusing the EN-ConvNTM module with the  $j$ -PPO method, the joint  $j$ -PPO+EN-ConvNTM algorithm is developed to significantly enhance adaptability to high-density mobile device scenarios. Leveraging two core modules—long-term spatiotemporal information extraction and continuous-discrete DRL algorithm optimization—the algorithm aims to maximize episode rewards while generating optimal UAV flight strategies, task offloading policies, and UAV charging schedules. This approach effectively mitigates multi-time-step errors

in information processing and significantly improve the equilibrium efficiency of UAV-assisted MEC systems.

- Numerical simulation results demonstrate that the proposed  $j$ -PPO+EN-ConvNTM algorithm exhibits robust applicability in high-density mobile device environments with varying numbers of UAVs and charging stations. Compared with existing baseline algorithms, the joint algorithm achieves significantly faster learning convergence and superior system performances.

The remainder of the paper is structured as follows. Initially, the related literature is explored in Section II. Subsequently, we define the system model and formulate the optimization problem in Section III. The novel algorithm is introduced in Section IV. Performance evaluations are presented in Section V, followed by the concluding remarks in Section VI.

## II. RELATED WORKS

The integration of UAVs into MEC, particularly in high-density user mobility scenarios, has become as a crucial research frontier. This integration aims to address the challenges of limited computing resources and the need for rapid data processing near end-users. Introduced to bring computational resources closer to data sources, MEC significantly reduces latency and network congestion [11]. In this context, UAVs have evolved beyond traditional surveillance roles to serve as mobile computing platforms [12].

The evolution of this field highlights a shift from traditional centralized cloud computing models to decentralized edge computing paradigms. Early works like Hu et al. [13] laid the foundation for MEC by proposing architectures for task offloading to edge servers. As demonstrated in studies such as Zeng et al. [14], the introduction of UAVs introduced a new dimension, enabling flexible, dynamic, and on-demand deployment. This transition is particularly crucial in high-density scenarios where user mobility and service demands fluctuate significantly. Research by Qin et al. [15] and Mohsen et al. [16] further advanced UAV-assisted MEC, exploring optimal deployment and resource allocation strategies.

However, recent advancements in UAV-assisted MEC have predominantly focused on scenarios with low user mobility and density [17]. This leaves a significant gap in research for high-density mobility environments, which are increasingly common in urban areas and large-scale events [18]. Current studies often overlook the dynamic nature of user demands and mobility patterns, which are critical factors in optimizing UAV-assisted MEC system performance [19]. In recent years, there has been a shift toward optimizing UAV-assisted MEC in high-density environments. Studies by Wang et al. [20] and Liu et al. [21] have investigated the challenges of resource allocation and task offloading in such scenarios, emphasizing the need for intelligent algorithms to effectively manage dynamic and heterogeneous demands. Addressing these challenges paves the way for the development of more robust, efficient, and scalable UAV-assisted MEC systems that can meet the evolving needs of high-density mobility applications.

It is noteworthy that in high-density user mobility scenarios, the primary challenge lies in effectively managing computational resources while environmental dynamic adaptability.

Existing frameworks and algorithms, which perform well in low-density environments, may not scale efficiently or adapt dynamically to rapid changes in user demand and mobility [22]. This necessitates the development of new algorithms and models that can predict user movement and adjust resource allocation in real-time. While studies such as those by Chen et al. [23] and McEnroe et al. [24] have started to address these issues, more comprehensive solutions are still required. The current research trend focuses on integrating advanced machine learning approaches into UAV-assisted MEC systems [25]. The intelligent method is expected to enhance the system's ability to adapt to user mobility and manage resources more efficiently in high-density scenarios. For instance, reinforcement learning and deep learning techniques could be utilized to predict user movement patterns and optimize UAV placement and flight trajectories accordingly [26], [27]. Additionally, the integration of 6G or secure technologies with UAV-assisted MEC is expected to further improve communication efficiency and computational capacity, offering new opportunities for high-density user management [28], [29].

The research develop of UAV-assisted MEC system illustrates the need for machine learning approaches that not only enable dynamic adjustment to changing user patterns and demands, but also address the challenges posed by high-density user mobility environments. Our work directly addresses this emerging challenge by proposing a novel deep reinforcement learning framework,  $j$ -PPO+EN-ConvNTM, specifically designed for high-density user mobility scenarios in UAV-assisted MEC systems. The  $j$ -PPO+EN-ConvNTM framework represents an advancement, incorporating state-of-the-art deep learning algorithms to optimize UAV flight trajectory and resource allocation. This approach aligns with the current shift towards machine learning in complex dynamic environment optimization.

By leveraging  $j$ -PPO, EN-ConvNTM, and STN, the framework is well-positioned to handle user movement and optimize the UAV flight trajectory and resource distribution in real-time.  $j$ -PPO enables efficient policy learning in complex dynamic environments, a key requirement in high-density user scenarios. EN-ConvNTM, a memory-augmented neural network with STN, enhances its capability by enabling the learning and recall of temporal patterns, essential for handling the user mobility and task offloading requirements [30]. This innovative combination in  $j$ -PPO+EN-ConvNTM not only addresses the current challenges in UAV-assisted MEC, but also establishes a new benchmark for future research, particularly in applying advanced machine learning to dynamic resource management.

### III. SYSTEM MODEL

In the scene of UAV-assisted MEC system shown as in Fig. 1, we examine the scenario involving fundamental cost-effective IoT devices, denoted as  $\mathcal{D} \triangleq \{1, 2, \dots, d, \dots, D\}$ . These IoT devices can autonomously determine their data offloading strategies. Offloaded data is then processed by UAVs, represented as  $\mathcal{U} \triangleq \{1, 2, \dots, u, \dots, U\}$ . Data that remains unoffloaded is partially processed by the embedded data processors within the IoT devices, depending on their inherent

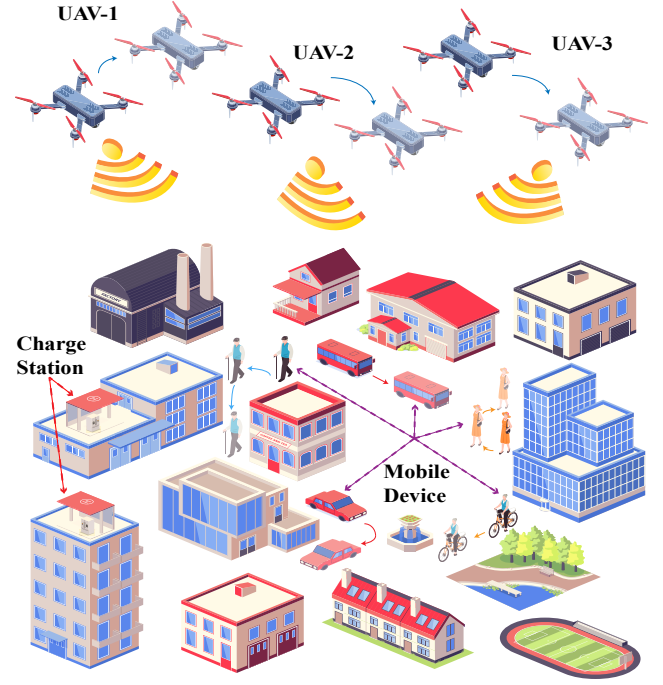


Fig. 1: The UAV-assisted MEC with mobile IoT devices.

processing capabilities. However, potential obstructions, such as high-rise buildings, denoted as  $\mathcal{J} \triangleq \{1, 2, \dots, j, \dots, J\}$ , exist and must be circumnavigated by the UAVs. Initially, each UAV has an origin energy level, and a series of charging stations, defined as  $\mathcal{Z} \triangleq \{1, 2, \dots, z, \dots, Z\}$ , are distributed to enable UAVs to recharge their energy reserves..

Next, we introduce the UAV-assisted MEC system from five aspects: IoT devices mobility model, channel model, delay and energy consumption, charging model, and UAV flying model. Finally, the optimization problem is formulated.

#### A. IoT Devices Mobility Model

It is assumed that all IoT devices are initially distributed randomly within their monitoring area, and the position of each device in the  $n$ th time slot is represented by  $Q_{d,n} = (x_{d,n}, y_{d,n})$ . The UAV is modeled as a broadcast MEC unit for computing offloading services, selecting IoT devices within a specified communication range for channel access. This allows devices to offload computational tasks to the UAV's onboard processor for edge processing.

Due to temporal dependencies in their movement, IoT device mobility adheres to physical motion laws, where current movement is influenced by historical trajectories. According to the Gauss-Markov (GM) mobility model, both the speed and direction of IoT device  $d$  can be modeled using two independent Gaussian distributions as [31]

$$v_{d,n} = \alpha v_{d,n-1} + (1 - \alpha) \bar{v}_{d,n} + \omega_{d,n} \sqrt{1 - \alpha^2}, \quad (1a)$$

$$\zeta_{d,n} = \beta \zeta_{d,n-1} + (1 - \beta) \bar{\zeta}_{d,n} + \varphi_{d,n} \sqrt{1 - \beta^2}. \quad (1b)$$

Here,  $v_{d,n}$  and  $\zeta_{d,n}$  respectively represent the mobility speed and the relative direction of the device at the time slot  $n - 1$ ,  $\bar{v}_{d,n}$  and  $\bar{\zeta}_{d,n}$  respectively denote the average speed and

absolute mobility direction, the coefficients  $\alpha$  and  $\beta$  are used to moderate the influence of previous states on the current state, with both values ranging between 0 and 1, and the variables  $\omega_{d,n}$  and  $\varphi_{d,n}$  are independent random Gaussian processes that effectively introduce stochastic variability into the device mobility's dynamics.

Therefore, the position information  $Q_{d,n}$  of the IoT device can be updated based on its dynamically changing speed  $v_{d,n-1}$  and direction  $\zeta_{d,n-1}$ . These updates incorporate the calculated values from the GM mobility model, where both speed and direction are adjusted according to the device's previous states and random Gaussian fluctuations. This ensures that the model accurately reflects the realistic mobility of IoT devices. We have

$$x_{d,n} = x_{d,n-1} + \tau v_{d,n-1} \cos \zeta_{d,n-1}, \quad (2a)$$

$$y_{d,n} = y_{d,n-1} + \tau v_{d,n-1} \sin \zeta_{d,n-1}. \quad (2b)$$

where  $\tau$  represents the time slot duration.

### B. Channel Model

It is presumed that the locations of IoT devices are known to the UAV, with the UAV's horizontal position denoted as  $Q_{u,n} = (x_{u,n}, y_{u,n}, h_u)$ , and here  $h_u$  is the UAV flight altitude. Given the dynamic nature of the UAV's actual flight environment, it is essential to regularly re-plan a feasible flight trajectory for each time slot. This re-planning strategy should be based on the real-time status and specific tasks of UAVs to enhance performance metrics. In our study, we focus on a scenario with high-density IoT devices. For each UAV, numerous ground devices with Line-of-Sight (LoS) channels are typically available at each time slot. UAVs will prioritize task offloading to ground devices with LoS channels. Thus, it is reasonable to make the assumption of an LoS channel model between the UAV and ground devices. The channel gain can be expressed as

$$g_{u,d,n} = \frac{\beta_0}{(x_{u,n} - x_{d,n})^2 + (y_{u,n} - y_{d,n})^2 + (h_u)^2}, \quad (3)$$

where  $\beta_0$  represents the channel coefficient at a reference distance of 1 meter.

Then, the data transmission rate from a IoT device to the UAV is determined by

$$R_{u,d,n}(p_d) = B \log_2 \left( 1 + \frac{g_{u,d,n} p_d}{N_0} \right), \quad (4)$$

where  $B$  indicates the bandwidth allocated to the IoT device,  $p_d$  denotes the transmit power, and  $N_0$  represents the noise power level. In our study, the OFDM technique is employed to enable task offloading for individual IoT devices, thereby minimizing the transmission interference among them. For the sake of simplicity, the transmission interference among devices is typically assumed to be ignored as existing works [5], [32].

### C. Delay and Energy Consumption

It is hypothesized that  $L_{d,n}$  denotes the total data volume for task processing in the  $n$ th time slot, and  $\lambda_{u,d,n}$  signifies the ratio of data offloaded from IoT device  $d$  to the UAV  $u$ .

The transmission delay and energy consumption of the IoT device to send data to the UAV are given by

$$T_{u,d,n}^{\text{Offload}} = \frac{\lambda_{u,d,n} L_{d,n}}{R_{u,d,n}(p_d)}, \quad (5a)$$

$$E_{u,d,n}^{\text{Offload}} = p_d T_{u,d,n}^{\text{Offload}}. \quad (5b)$$

The delay and energy costs associated with local computing are respectively described by

$$T_{d,n}^{\text{Local}} = \frac{(1 - \sum_U \lambda_{u,d,n}) L_{d,n} C_d}{f_d}, \quad (6a)$$

$$E_{d,n}^{\text{Local}} = \eta f_d^3 T_{d,n}^{\text{Local}}, \quad (6b)$$

where  $C_d$  represents the CPU cycles necessary to process 1 bit of input data,  $f_d$  represent the CPU frequency of the  $d$ th IoT device, and  $\eta$  indicates the effective capacitance coefficient.

Then, the computation delay experienced by the UAV in processing the tasks offloaded from the IoT device is computed as

$$T_{u,d,n}^{\text{Compute}} = \frac{\lambda_{u,d,n} L_{d,n} C_u}{f_u}, \quad (7)$$

where  $C_u$  and  $f_u$  respectively represent the required CPU cycles and frequency of UAV for the offloaded data processing.

Similarly, the energy consumption incurred by the UAV for offloaded task processing is

$$E_{u,d,n}^{\text{Compute}} = \mu f_u^3 T_{u,d,n}^{\text{Compute}}. \quad (8)$$

where  $\mu$  indicates the effective capacitance coefficient of UAV.

Upon completing the task computations, the UAV sends the results back to the respective IoT device. Let  $\tilde{L}_{d,n}$  represent the number of bits transmitted back the IoT device. Consequently, the transmission delay from the UAV to device  $d$  is given by

$$T_{u,d,n}^{\text{Transmit}} = \frac{\lambda_{u,d,n} \tilde{L}_{d,n}}{R_{u,d,n}(p_u)}. \quad (9)$$

where  $p_u$  is the transmit power.

The relevant energy consumption can be derived as

$$E_{u,d,n}^{\text{Transmit}} = p_u T_{u,d,n}^{\text{Transmit}}. \quad (10)$$

Thus, the cumulative energy consumption for both the UAV and the IoT device, encompassing data processing, is calculated as

$$E_{d,n} = \sum_U \left( p_d T_{u,d,n}^{\text{Offload}} + \mu f_u^3 T_{u,d,n}^{\text{Compute}} + p_u T_{u,d,n}^{\text{Transmit}} \right) + \eta f_d^3 T_{d,n}^{\text{Local}}. \quad (11)$$

### D. Charging Model

The UAV can decide whether or not to charge. If UAV decides to charge, UAV will fly to an effective area near the charging station (obviously the action will consume energy) and recharge a partial of energy. The energy charging process is given by [10]

$$E_{u,n}^{\text{Old-Left}} = E_{u,n-1}^{\text{Left}} - E_{u,n}^{\text{Fly}}, \quad (12a)$$

$$E_{u,n}^{\text{Left}} = E_{u,n}^{\text{Old-Left}} + E_{u,n}^{\text{Charge}} = E_{u,n}^{\text{Old-Left}} + q \xi_{u,n} E_{u,n}^{\text{max}}, \quad (12b)$$

where  $\xi_{u,n} \in \{0, 1\}$  is the charging indicator with  $\xi_{u,n} = 1$  signifying UAV in charging state,  $q$  represents the charging ratio,  $E_{u,n}^{\text{Old-Left}}$ ,  $E_{u,n}^{\text{Left}}$ ,  $E_{u,n}^{\text{Charge}}$ , and  $E_{u,n}^{\text{max}}$  represent the UAV's energy before charging, after charging, charging energy, and energy capacity, respectively.

### E. UAV Flying Model

In our study, for simplicity, we assume that the UAVs maintain a constant speed  $v_u$  during task offloading and hover when charging. As a result, there are only two scenarios, and the energy consumption of the UAV is directly related to the flying time and flying speed. The energy consumed by the UAV during a single time slot is given by [33]

$$E_{u,n}^{\text{Fly}} = \begin{cases} P^{\text{Charge}}\tau, & \text{if Charging,} \\ P(v_u)\tau, & \text{Others,} \end{cases} \quad (13)$$

where  $P(v_u)$  and  $P^{\text{Charge}}$  respectively denote the UAV's flying power and hovering power when charging.

### F. Problem Formulation

Our proposed framework aims to achieve the following objectives: (1) To prioritize energy conservation across the entire task offloading period. It is recommended to utilize nearby UAVs for data acquisition to reduced energy consumption. (2) To efficiently transmit and process data from devices through optimal UAV path planning. (3) To ensure maximum variance in the data collected by all UAVs, thereby achieving a balanced geographical distribution among devices.

First, an energy consumption coefficient,  $\kappa_n$ , is defined as

$$\kappa_n = \frac{\eta_u \sum_U \sum_{i=1}^n E_{u,i}^{\text{Fly}} + \eta_d \sum_D \sum_{i=1}^n E_{d,i}}{\eta_u \sum_U \sum_{i=1}^n E_{u,i}^{\text{Charge}} + \eta_u \sum_U E_u^{\text{Origin}} + \eta_d \sum_D E_d^{\text{Origin}}}, \quad (14)$$

where  $\eta_u$  and  $\eta_d$  are energy balance coefficient,  $E_u^{\text{Origin}}$  and  $E_d^{\text{Origin}}$  respectively represent the origin energy levels of UAVs and IoT devices. It is obvious that a smaller value of  $\kappa_n$  implies higher energy consumption efficiency.

To capture data diversity, the spatial equity index  $f_n$ , is introduced to measure the fairness of territorial coverage by UAVs as

$$f_n = \frac{(\sum_D \iota_{d,n})^2}{D \sum_D (\iota_{d,n})^2}, \quad (15)$$

where  $\iota_{d,n}$  represents the total number of times that an IoT device  $d$  has been visited by all UAVs till time slot  $n$ . Clearly, evenly collecting data from all IoT devices ensures a high level of geographical fairness.

Additionally, the data collection coefficient  $\psi_n$  is defined as

$$\psi_n = \frac{\sum_D \sum_U \sum_{i=1}^n \lambda_{u,d,i} L_{d,i}}{\sum_D \sum_{i=1}^n L_{d,i}}. \quad (16)$$

A higher data collection coefficient indicates that more data of IoT devices has been collected.

The objective of optimization problem formulation for UAV-assisted MEC is to achieve the minimum energy expenditure coefficient and maximum data collection coefficient and fair-

ness by optimally designing the UAV trajectory, offloading strategy, and charging indicator, which can be built as

$$\begin{aligned} \mathbf{P1} : \quad & \max_{Q, \lambda, \xi} \Omega_n(\psi_n, f_n, \kappa_n) \\ \text{s.t.} \quad & \text{C1 : } \|Q_{u,n+1} - Q_{u,n}\| \leq D_{\max}, \quad \forall u, \quad (17a) \\ & \text{C2 : } \lambda_{u,d,n} \in [0, 1], \quad \forall u, \forall d, \quad (17b) \\ & \text{C3 : } \xi_{u,n} \in [0, 1], \quad \forall u, \quad (17c) \\ & \text{C4 : } \sum_U \lambda_{u,d,n} \leq 1, \quad \forall d, \quad (17d) \\ & \text{C5 : } E_{u,n}^{\text{Left}}, E_{u,n}^{\text{Old-Left}} \geq 0, \quad \forall u, \quad (17e) \end{aligned}$$

where  $Q = \{Q_{u,n}\}$ ,  $\lambda = \{\lambda_{u,d,n}\}$ ,  $\xi = \{\xi_{u,n}\}$ ,  $D_{\max}$  is the maximum flight distance of UAV in one time slot, and the overall equilibrium efficiency  $\Omega_n(\psi_n, f_n, \kappa_n)$  can be modeled as a multi-objective-based metric, such as a multiplication-division-base cost function  $\Omega_n = \psi_n f_n / \kappa_n$  and a weighted cost function  $\Omega_n = \beta_1 \psi_n + \beta_2 f_n - \beta_3 \kappa_n$ . These functions are relevant to different application scenarios. Here  $\beta_1$ ,  $\beta_2$ , and  $\beta_3$  are weighting factors.

### IV. $j$ -PPO+EN-CONVNTM FRAMEWORK

In addressing the optimization problem **P1**, the application of DRL is particularly apt due to several intrinsic characteristics of the problem and the strengths of DRL methodologies. Firstly, the problem presents a high-dimensional action and state space, as evidenced by the multiple variables, such as  $Q$ ,  $\lambda$ , and  $\xi$ , and their inter-dependencies. DRL excels in navigating these complex, multi-variable environments. Secondly, the dynamic nature and sequential decision-making inherent in the problem, as highlighted by the relationship between  $Q_{u,n+1}$  and  $Q_{u,n}$ , align well with DRL's proficiency in learning optimal strategies over a series of actions in fluctuating circumstances. Furthermore, the problem's requirement for multi-objective optimization is well-suited to DRL's capability to balance multiple goals through intricate reward function learning. Additionally, the presence of both continuous (like  $Q$ ) and discrete variables (such as  $\lambda$  and  $\xi$ ) in the problem is well within the purview of DRL, which can handle mixed types of action spaces. Finally, the complexity introduced by the multiple constraints (from C1 to C5) can be effectively managed by DRL, as it can learn policies that not only adhere to these constraints but also optimize performance within their bounds. Therefore, the utilization of DRL in solving this optimization problem is justified by its flexibility and adaptive capabilities in high-dimensional, dynamic environments, and its proficiency in handling complex, multi-objective, and constrained optimization scenarios.

#### A. Joint Continuous and Discrete Action Decision: $j$ -PPO

In this paper, we propose a DRL framework termed “ $j$ -PPO+EN-ConvNTM”, which integrates long-sequence spatiotemporal data modeling (referred to as “EN-ConvNTM”) and a joint continuous and discrete action decision-making module (named “ $j$ -PPO”), as illustrated in Fig. 2. Specifically, EN-ConvNTM is designed to encode a 3-D history vector that meticulously tracks the flight paths of UAVs. Utilizing this 3-D history vector, the  $j$ -PPO framework actively manages the

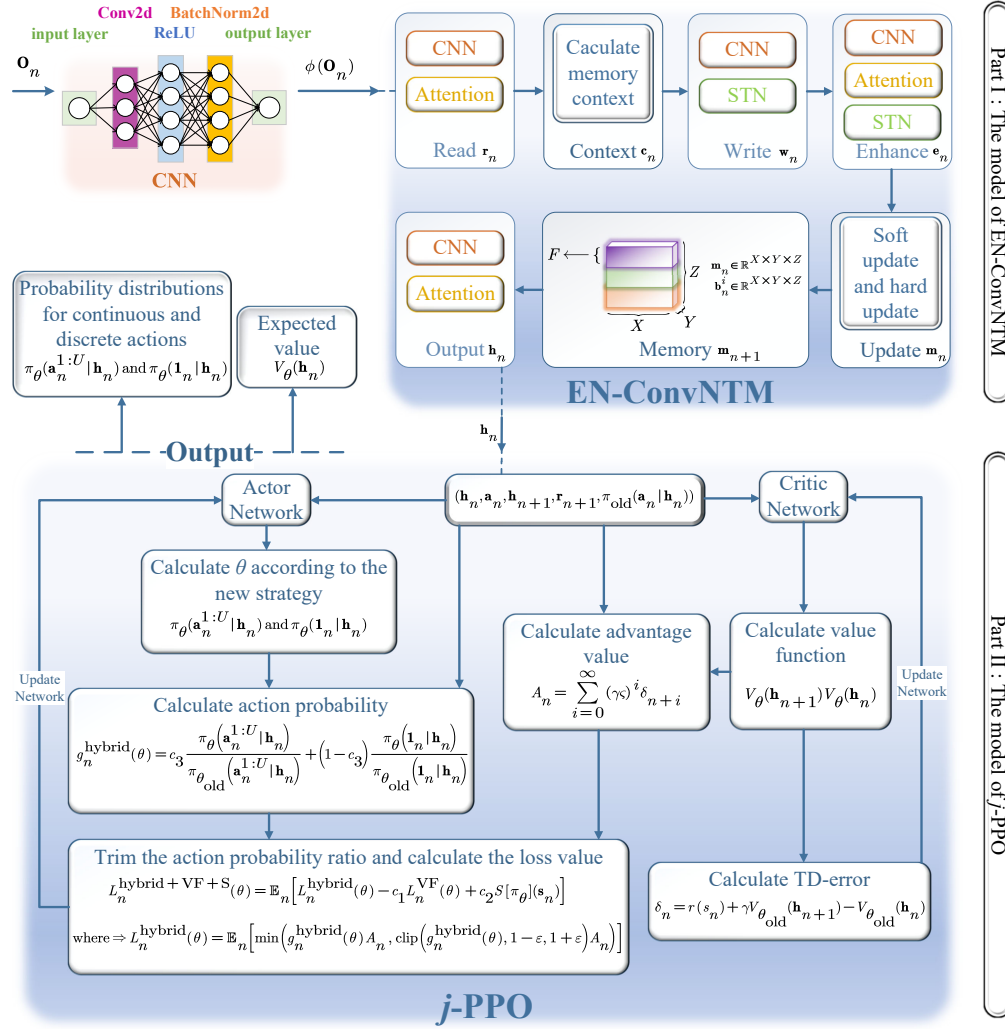


Fig. 2:  $j$ -PPO+EN-ConvNTM framework

UAVs' flight paths while making precise decisions regarding communication and battery recharging.

Below, we first outline the definition of the partially observed Markov decision process (POMDP) as applied to the UAV-assisted MEC system. Subsequently, the method for long-sequence spatiotemporal data modeling, referred to as "EN-ConvNTM", is introduced for preprocessing. Finally, the " $j$ -PPO" module is employed to address the optimization problem through joint continuous and discrete action decision-making.

### B. POMDP Definition

To tackle the aforementioned optimization problem, it is necessary to develop a POMDP framework instead of a fully observable MDP. This is because the system's dynamics are inherently complex and uncertain. The POMDP approach is particularly suitable for environments where the state information is not fully observable, which is highly relevant to our problem involving interactions between UAVs and IoT devices. Factors such as precise UAV locations or internal states of IoT devices may be partially unknown or obscured.

Furthermore, the POMDP can accommodate the inherent uncertainty in action outcomes and observations, which is crucial in unpredictable scenarios such as UAV flight decision-making or IoT data transmission. Unlike MDPs that assume complete state information during decision-making, POMDPs use belief states, a probabilistic representation of the possible states of the environment, to guide decision-making under uncertainty. This approach effectively captures the real-world complexities of partial observability and decision-making under uncertainty, making POMDP a more fitting and realistic framework for optimizing operational strategies in dynamic and partially known environments involving UAV and IoT interactions.

The POMDP framework is characterized by a tuple  $(\mathcal{S}, \mathcal{A}, r, \rho_0, \rho, \mathcal{O}, P_r, \gamma)$ , where  $\mathcal{S}$  represents the set of environment states,  $\mathcal{A}$  is the set of actions, and  $\gamma$  is the discount factor. The reward function is expressed as  $r(s_n)$ . The initial state of the environment is given by the distribution function  $\rho_0(s_0)$ , with  $\rho(s_{n+1}|s_n, \mathbf{a}_n)$  describing the state transition function. In this partially observable setting, the agent receives observations  $\mathbf{o}_n$  from  $\mathcal{O}$  rather than direct access to state  $s_n$ . The probability of observing  $\mathbf{o}_n$  given  $s_n$  is defined by  $P_r(\mathbf{o}_n|s_n)$ .



Subsequently, based on the system model and the optimal control challenge, further analysis is conducted on the POMDP of the MEC system. The specific observation, action, and reward function are designed as follows.

1) *Observation*: The observation  $\mathbf{o}_n$  is conceptualized as a 3-D vector including 3 channels. This vector maps the 2-D real-world locations of obstacles, charging stations, IoT devices, and UAVs into these channels.

In the first channel, which details the geographical layout, the positions of obstacle are assigned a value of -1, while the positions of IoT devices are filled with values representing their current remaining data.

$$o_n^{\text{1st Ch.}} : (\phi(x_{d,n}), \phi(y_{d,n}), 1) = \begin{cases} L_{d,n}, \\ -1. \end{cases} \quad (18)$$

The second channel illustrates the geographic layout of charging stations and UAVs. In this layer, the positions occupied by UAVs indicate their current remaining energy levels, while the positions of charging stations are marked with a value of -1.

$$o_n^{\text{2nd Ch.}} : (\phi(x_{u,n}), \phi(y_{u,n}), 2) = \begin{cases} E_{u,n}^{\text{Left}}, \\ -1. \end{cases} \quad (19)$$

In the third channel, the grid positions corresponding to IoT devices show the number of visits by all UAVs up to the current time slot, while all other values are set to zero.

$$o_n^{\text{3rd Ch.}} : (\phi(x_{d,n}), \phi(y_{d,n}), 3) = \begin{cases} \iota_{d,n}, \\ 0. \end{cases} \quad (20)$$

2) *Action*: In UAV-assisted MEC problem, the optimization variables include continuous variables such as  $Q$ , as well as discrete variables such as  $\lambda$  and  $\xi$ . This belongs to a joint continuous and discrete action decision-making problem. Thus, the action is characterized as a tuple

$$\mathbf{a}_n = (\mathbf{a}_n^1, \mathbf{a}_n^2, \dots, \mathbf{a}_n^U, \mathbf{1}_n), \quad (21)$$

where the discrete variables are denoted by an indicator vector function  $\mathbf{1}_n$ , and the continuous action  $\mathbf{a}_n^u$  indicates UAV flight trajectories. This action function will control the UAV either to charge or to act as a offloading MEC data collector.

3) *Reward Function*: The reward function is defined as

$$r_n = \Omega_n - p_{u,n}. \quad (22)$$

Here, the reward function is totally determined by the multi-objective-based cost function. Additionally, a penalty  $p_{u,n}$  is typically introduced when the distance between two UAVs is too small, or when a UAV is too close to an obstacle, or when a UAV is close to depleting its energy reserves [6].

### C. EN-ConvNTM Module Preprocessing

In the domain of POMDP, an integrator is essential to condense the sequence of observations  $(\mathbf{o}_0, \mathbf{o}_1, \dots, \mathbf{o}_{n-1}, \mathbf{o}_n)$  into a comprehensive representation  $\mathbf{h}_n$ . Each observation encapsulates the positional data of UAVs, obstacles, charging stations, and IoT devices.

Traditional integrators, such as recurrent neural networks (RNNs) and long short-term memorys (LSTMs), while being

quite effective, often struggle to fully capture the structural complexity of the data, particularly when dealing with multi-dimensional inputs. ConvLSTM represents a viable alternative that preserves structural information; however, its capacity to integrate information over long sequences may be limited. To address these challenges, we introduce an enhanced version of the ConvNTM integrator, EN-ConvNTM, equipped with an external multi-dimensional memory and integrated with a STN. This improved ConvNTM excels at encoding spatiotemporal information across extended sequences, ensuring that the spatial relationships within the data are effectively captured and utilized.

The EN-ConvNTM module is designed to interpret actions  $\mathbf{a}_n$  from sequences of observations  $\mathbf{o}_n$  and  $\mathbf{o}_{n+1}$ , utilizing 3-D vectors  $(\mathbf{o}_1, \dots, \mathbf{o}_n)$  to construct a historical trajectory. When these observations are considered from a temporal perspective, it reveals the UAV trajectories and temporal interactions among all entities. These aspects are as crucial as spatial information for cooperative UAV operations and decision-making. The trajectories of UAVs derived from extended spatiotemporal history provide invaluable references for modeling the intricate dynamics within the realistic environment. This approach is augmented with spatial transformation capabilities and an advanced memory mechanism to enrich the spatiotemporal understanding of the data. The EN-ConvNTM module can be expressed as

$$\mathbf{h}_n = \text{EN-ConvNTM}(\text{STN}(\phi(\mathbf{o}_n))). \quad (23)$$

The STN module, denoted as  $\text{STN}(\cdot)$ , applies a spatial transformation to the input  $\phi(\mathbf{o}_n)$  before it is processed by the ConvNTM. This ensures that the network focuses on the most relevant spatial features of the observation. The output  $\mathbf{h}_n$  is then a 3-D vector that encapsulates the spatiotemporally relevant information, which has been enhanced through spatial transformation.

The components and operations of the EN-ConvNTM are described as follows, with each component being differentiable and trainable via gradient descent as

$$\mathbf{m}_n = [\mathbf{b}_n^1, \mathbf{b}_n^2, \dots, \mathbf{b}_n^L]. \quad (24)$$

Here,  $[\cdot]$  denotes the concatenation of 3-D vectors, where a 3-D block  $\mathbf{b}_n^i$  represents the memory unit, and the complete memory  $\mathbf{m}_n$  is formed, and  $L$  is the length of memory blocks.

The detailed operations of EN-ConvNTM are as follows.

$$\mathbf{r}_n = \text{read}(\mathbf{m}_n), \quad (25a)$$

$$\mathbf{c}_n = \text{context}(\mathbf{r}_n, \mathbf{m}_n, \text{STN}(\phi(\mathbf{o}_n))), \quad (25b)$$

$$\mathbf{w}_n = \text{write}(\mathbf{r}_n, \mathbf{c}_n, \text{STN}(\phi(\mathbf{o}_n))), \quad (25c)$$

$$\mathbf{e}_n = \text{enhancement}(\mathbf{r}_n, \mathbf{w}_n, \text{STN}(\phi(\mathbf{o}_n))), \quad (25d)$$

$$\mathbf{m}_{n+1} = \text{update}((\mathbf{w}_n + \mathbf{e}_n), \mathbf{m}_n), \quad (25e)$$

$$\mathbf{h}_n = \text{output}(\mathbf{r}_n, \mathbf{c}_n, \text{STN}(\phi(\mathbf{o}_n))). \quad (25f)$$

This EN-ConvNTM structure enables a deeper and more contextually aware representation of the environment, which is critical for making informed decisions in complex and dynamic scenarios. The detailed information for operation about read, context, write, update, and output can be found in [10].

The enhancement operation within the ConvNTM architecture integrates feature amplification and information distillation to optimize the representation of critical information.

Feature amplification begins by applying a spatial transformation to the input  $\phi(\mathbf{o}_n)$  using STN, which refines the input to highlight operationally significant regions. This is further enriched by integrating contextual embeddings  $\mathbf{w}_n$ , which capture the broader dependencies within the dataset, ensuring a more comprehensive understanding of the underlying environment.

Building on this, information distillation employs an attention mechanism to selectively emphasize elements with the highest informational value, sharpening the model's focus on the most relevant data segments. Additionally, dimensionality reduction techniques are applied to streamline the feature set, effectively crystallizing the knowledge representation while minimizing redundancy. Together, these processes synthesize an enhanced vector  $\mathbf{e}_n$  that encapsulates salient features and eliminates extraneous details, contributing to a more robust and distilled representation.

Based on feature amplification and information distillation, the enhancement operational formula is given by

$$\mathbf{e}_n = \psi^E([\text{STN}(\phi(\mathbf{o}_n)), \mathbf{w}_n, \text{Attention}(\mathbf{W}^{\text{att}} * [\phi(\mathbf{r}_n), \mathbf{w}_n])]), \quad (26)$$

where  $\text{Attention}(\cdot)$  applies a weighted importance to the features,  $\mathbf{W}^{\text{att}}$  is the associated weighting matrix, and  $\psi^E(\cdot)$  involves a combination of spatial transformation, contextual embedding, and attention mechanism to refine and amplify the memory representation.

The computational steps of enhancement operation are as follows.

1) Combining spatial transformation and contextual embedding:

$$\mathbf{f}_{\text{combined}} = [\text{STN}(\phi(\mathbf{o}_n)) \oplus \mathbf{w}_n]. \quad (27)$$

Here,  $\text{STN}(\phi(\mathbf{o}_n))$  applies a spatial transformation to the input  $\phi(\mathbf{o}_n)$ , optimizing its spatial features to highlight operationally critical regions. This output is then concatenated (denoted by  $\oplus$ ) with contextual embedding  $\mathbf{w}_n$ , which encapsulate the contextual dependencies within the dataset.

2) Applying attention weighting:

$$\mathbf{f}_{\text{att}} = \text{Attention}(\mathbf{W}^{\text{att}} * \mathbf{f}_{\text{combined}}). \quad (28)$$

The attention mechanism assesses and amplifies the most important elements within the enhanced dataset by applying a weighted importance using the weight matrix  $\mathbf{W}^{\text{att}}$ . The result is  $\mathbf{f}_{\text{att}}$ , an attention-weighted feature vector that sharpens the focus on segments with maximal informational content.

3) Feature fusion and information distillation:

$$\mathbf{e}_n = \sigma(\mathbf{W}^e * [\mathbf{f}_{\text{att}} \oplus \mathbf{f}_{\text{combined}}] + \mathbf{b}^e). \quad (29)$$

In this step,  $\mathbf{W}^e$  and  $\mathbf{b}^e$  represent the weights and biases of a linear layer, respectively, and  $\sigma$  is a nonlinear activation function, such as ReLU or tanh. The purpose of this step is to further integrate and distill the features, producing the final enhanced vector  $\mathbf{e}_n$  by combining both attention-weighted and spatially-contextual features.

This resulting vector  $\mathbf{e}_n$  equips the EN-ConvNTM with a memory substrate for subsequent operations, reinforcing the system's capacity for insightful data interpretation and complex decision-making.

#### D. Joint Continuous and Discrete Action Decision: $j$ -PPO

According to the POMDP definition, the long-term cumulative rewards called return is given by

$$R_n = \sum_{i=0}^n \gamma^i r(s_i), \quad (30)$$

where  $\gamma \in (0, 1)$  is the discount factor, controlling the weight of future rewards in the decision-making process.

The objective is to develop a policy  $\pi_\theta(\mathbf{a}_n|\mathbf{h}_n)$ , where history information is encapsulated in  $\mathbf{h}_n$ . This stochastic policy  $\pi_\theta(\mathbf{a}_n|\mathbf{h}_n)$  aims to maximize

$$\mathbb{E}_{(\mathbf{s}_0, \mathbf{o}_0, \mathbf{a}_0, \dots)}[R_n], \quad (31)$$

where  $\mathbf{s}_0 \sim \rho_0(\mathbf{s}_0)$ ,  $\mathbf{o}_n \sim P_r(\mathbf{o}_n|\mathbf{s}_n)$ ,  $\mathbf{a}_n \sim \pi_\theta(\mathbf{a}_n|\mathbf{h}_n)$ , and  $\mathbf{s}_{n+1} \sim \rho(\mathbf{s}_{n+1}|\mathbf{s}_n, \mathbf{a}_n)$ .

Simultaneously, the value function is defined as follows.

$$V_{\pi_\theta}(\mathbf{h}_n) = \mathbb{E}_{\mathbf{s}_n|\mathbf{h}_n}[r(\mathbf{s}_n)] + \mathbb{E}_{(\mathbf{a}_n, \mathbf{s}_{n+1}, \mathbf{o}_{n+1}, \mathbf{a}_{n+1}, \dots)|\mathbf{h}_n}[G_n]. \quad (32)$$

where  $G_n = \sum_{i=1}^{\infty} \gamma^i r(\mathbf{s}_{n+i})$ .

Typically, policy gradient methods adjust the probabilities of actions to increase those leading to higher returns and decrease those leading to lower returns, converging on an optimal policy. This approach employs a stochastic gradient ascent algorithm, with the gradient estimator derived by differentiating the objective as follows.

$$L_n(\theta) = \mathbb{E}_n[\log \pi_\theta(\mathbf{a}_n|\mathbf{h}_n) A_n], \quad (33)$$

Here,  $\mathbb{E}_n$  represents the empirical average computed from a finite batch of samples, and  $A_n$  serves as an generalized advantage estimation (GAE) defined as

$$A_n = \sum_{i=0}^{\infty} (\gamma \varsigma)^i \delta_{n+i}, \quad (34)$$

where  $\varsigma$  is the GAE hyperparameter, and

$$\delta_n = r(s_n) + \gamma V_{\theta_{\text{old}}}(\mathbf{h}_{n+1}) - V_{\theta_{\text{old}}}(\mathbf{h}_n), \quad (35)$$

and here  $V_\theta$  represents the value function, which is parameterized by  $\theta$  updated as

$$\theta \leftarrow \theta + l_r \nabla_\theta L_n(\theta), \quad (36)$$

where  $l_r$  is the learning rate, and  $\nabla$  is the gradient operator.

The clipped version of PPO employs a unique clipping mechanism within the objective function to discourage significant deviations from the old policy, rather than incorporating a KL-divergence term and a constraint. The objective of clipped PPO is

$$L_n^{\text{CLIP}}(\theta) = \mathbb{E}_n[\min(g_n(\theta) A_n, \text{clip}(g_n(\theta), 1 - \varepsilon, 1 + \varepsilon) A_n)], \quad (37)$$



and

$$g_n(\theta) = \frac{\pi_\theta(\mathbf{a}_n | \mathbf{h}_n)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_n | \mathbf{h}_n)}. \quad (38)$$

Here, the first term within the  $\min(\cdot)$  function acts as a “surrogate” objective, where  $g_n(\theta)$  represents the probability ratio. The second term adjusts this surrogate by clipping the probability ratio, ensuring that the chosen objective, either clipped or unclipped, is the lower bound of the unclipped version.

Furthermore, when leveraging a neural network architecture that shares parameters across both the policy and value function, the clip loss alone is insufficient. To promote adequate exploration, the loss function should integrate the policy surrogate with a term for value function error and an entropy bonus as

$$L_n^{\text{CLIP+VF+S}}(\theta) = \mathbb{E}_n \left[ L_n^{\text{CLIP}}(\theta) - c_1 L_n^{\text{VF}}(\theta) + c_2 S[\pi_\theta](s_n) \right], \quad (39a)$$

$$L_n^{\text{VF}}(\theta) = \mathbb{E}_n \left[ \left( V_\theta(\mathbf{h}_n) - V_{\pi_{\theta_{\text{old}}}}(\mathbf{h}_n) \right)^2 \right], \quad (39b)$$

where  $c_1$  and  $c_2$  respectively denote the loss coefficient and entropy coefficient,  $S$  represents an entropy bonus, and  $L_n^{\text{VF}}$  refers to the error in the value function.

Given that our scenario includes both continuous actions for UAV flight and discrete actions for data offloading or battery charging, we start with PPO and adapt its loss functions to suit the requirements. This modified version is henceforth referred to as “ $j$ -PPO”:

$$L_n^{\text{hybrid+VF+S}}(\theta) = \mathbb{E}_n \left[ L_n^{\text{hybrid}}(\theta) - c_1 L_n^{\text{VF}}(\theta) + c_2 S[\pi_\theta](s_n) \right], \quad (40a)$$

$$L_n^{\text{hybrid}}(\theta) = \mathbb{E}_n \left[ \min \left( g_n^{\text{hybrid}}(\theta) A_n, \text{clip} \left( g_n^{\text{hybrid}}(\theta), 1 - \varepsilon, 1 + \varepsilon \right) A_n \right) \right], \quad (40b)$$

$$g_n^{\text{hybrid}}(\theta) = c_3 \frac{\pi_\theta(\mathbf{a}_n^{1:U} | \mathbf{h}_n)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_n^{1:U} | \mathbf{h}_n)} + (1 - c_3) \frac{\pi_\theta(\mathbf{1}_n | \mathbf{h}_n)}{\pi_{\theta_{\text{old}}}(\mathbf{1}_n | \mathbf{h}_n)}. \quad (40c)$$

Here,  $c_3 \in (0, 1)$  serves as a weighting factor. In “ $j$ -PPO”, the original probability ratio  $\pi_\theta(\mathbf{a}_n | \mathbf{h}_n) / \pi_{\theta_{\text{old}}}(\mathbf{a}_n | \mathbf{h}_n)$  used in PPO is substituted by a hybrid formulation. Notably, as detailed in Eqn. (40c), the hybrid probability ratio  $g_n^{\text{hybrid}}(\theta)$  merges the continuous action probability ratio  $\pi_\theta^{\text{cont}}(\mathbf{a}_n^{1:U} | \mathbf{h}_n) / \pi_{\theta_{\text{old}}}^{\text{cont}}(\mathbf{a}_n^{1:U} | \mathbf{h}_n)$  with the discrete action probability ratio  $\pi_\theta^{\text{disc}}(\mathbf{1}_n | \mathbf{h}_n) / \pi_{\theta_{\text{old}}}^{\text{disc}}(\mathbf{1}_n | \mathbf{h}_n)$ . Directly maximizing  $g_n^{\text{hybrid}}(\theta)$  could result in overly large policy updates, hence the introduction of the  $\text{clip}(g_n^{\text{hybrid}}(\theta), 1 - \varepsilon, 1 + \varepsilon)$  term to temper the surrogate objective by clipping the probability ratio. Furthermore, by taking the minimum of both the clipped and unclipped objectives, we ensure that the final objective remains a lower bound on the unclipped objective as shown in Eqn. (40b).

Therefore, the policy and expected value derived from the  $j$ -PPO algorithm can be summarized as

$$\pi_\theta^{\text{cont}}(\mathbf{a}_n^{1:U} | \mathbf{h}_n) = j\text{-PPO}_\theta^{\text{cont}}(\mathbf{h}_n), \quad (41a)$$

$$\pi_\theta^{\text{disc}}(\mathbf{1}_n | \mathbf{h}_n) = j\text{-PPO}_\theta^{\text{disc}}(\mathbf{h}_n), \quad (41b)$$

$$V_\theta(\mathbf{h}_n) = j\text{-PPO}_\theta(\mathbf{h}_n). \quad (41c)$$

The  $j$ -PPO functions utilize this enriched representation for decision-making, mapping the 3-D historical data to decisions and value assessments.

---

#### Algorithm 1: The $j$ -PPO+EN-ConvNTM Algorithm

---

```

1: for iteration = 1, 2, ... do
2:   for i = 1, 2, ...,  $\tilde{N}$  do
3:     for n = 1, 2, ..., N do
4:       Obtain observation  $\mathbf{o}_{n,i}$  from environment i.
5:       Calculate continuous and discrete action
         distributions  $\pi_{\theta_{\text{old}}}^{\text{cont}}(\mathbf{a}_{n,i}^{1:U} | \mathbf{h}_{n,i})$  and  $\pi_{\theta_{\text{old}}}^{\text{disc}}(\mathbf{1}_{n,i} | \mathbf{h}_{n,i})$ 
         using Eqns. (23), (41a), (41b).
6:       Sample a combined action  $\mathbf{a}_{n,i} = (\mathbf{a}_{n,i}^{1:U}, \mathbf{1}_{n,i})$ 
         from the above distributions.
7:       Execute  $\mathbf{a}_{n,i}$  to obtain the reward  $r_{n,i}$ .
8:       Derive advantage estimates  $A_{1:N,i}$  using Eqns.
         (32), (34), (35), (39b), and (41c).
9:       Assemble trajectory  $\chi_{1:N,i}$ .
10:    end for
11:  end for
12:  Segment trajectories  $\chi_{1:N,1:\tilde{N}}$  into
     $\mathcal{T} = \{\chi_{1:K,1:\tilde{N}}, \chi_{2:(K+1),1:\tilde{N}}, \dots, \chi_{(N-K):N,1:\tilde{N}}\}$ .
13:  Optimize  $L_n^{\text{hybrid+VF+S}}$  according to Eqns. (40a),
    (40b), and (40c) with respect to  $\theta$  for  $K$  epochs, using
    the mini-batch of size  $M \leq N(N - K)$  and a learning
    rate  $l_r$ .
14:  Update  $\theta_{\text{old}}$  to  $\theta$ :  $\theta_{\text{old}} \leftarrow \theta$ .
15: end for
```

---

#### E. $j$ -PPO+EN-ConvNTM Algorithm

As detailed in proposed  $j$ -PPO+EN-ConvNTM as algorithm, each training iteration begins by running  $\tilde{N}$  episodes with the parameters set to  $\theta_{\text{old}}$  (Line 2). These episodes consist of  $N$  time steps each (Line 3). At every time step  $n$ , the agent acquires an observation  $\mathbf{o}_{n,i}$  from environment (Line 4). Utilizing Eqns. (23), (41a), and (41b), the continuous and discrete action distributions,  $\pi_{\theta_{\text{old}}}^{\text{cont}}(\mathbf{a}_{n,i}^{1:U} | \mathbf{h}_{n,i})$  and  $\pi_{\theta_{\text{old}}}^{\text{disc}}(\mathbf{1}_{n,i} | \mathbf{h}_{n,i})$ , are determined (Line 5). An action is then sampled from these distributions (Line 6), and executed within environment  $i$  to earn a corresponding reward (Line 7). Following by this, the advantage estimates  $A_{1:n,i}$  are calculated using Eqns. (32), (34), (35), (39b), and (41c) (Line 8). Subsequently, a complete trajectory for episode  $i$ , defined as  $\chi_{1:N,i} = (\mathbf{o}_{1:N,i}, \mathbf{a}_{1:N,i}, V_{\pi_{\theta_{\text{old}}}}(\mathbf{h}_{1:N,i}), \pi_{\theta_{\text{old}}}^{\text{cont}}(\mathbf{a}_{n,i}^{1:U} | \mathbf{h}_{1:N,i}), \pi_{\theta_{\text{old}}}^{\text{disc}}(\mathbf{1}_{1:N,i} | \mathbf{h}_{1:N,i}), A_{1:N,i}, \mathbf{m}_{0:N-1,i})$ , is collected (Line 9).

Importantly, when integrating an on-policy DRL framework with EN-ConvNTM, the sequential use of samples is required, greatly enhancing their correlation. To mitigate this, we divide the extended sample sequences  $\chi_{1:N,1:\tilde{N}}$  into smaller segments  $\{\chi_{1:K,1:\tilde{N}}, \chi_{2:(K+1),1:\tilde{N}}, \dots, \chi_{(N-K):N,1:\tilde{N}}\}$ , where each segment is of length  $K$  (Line 12). In the loss function,  $M$  segments are randomly chosen to refine the surrogate  $L_n^{\text{hybrid+VF+S}}$  at each time step  $n$  (Line 13). This method not only reduces sample correlation and accelerates training but also preserves the full historical trajectory for later analysis during testing. Ultimately, we update  $\theta_{\text{old}}$  to  $\theta$  (Line 14) before proceeding to the next iteration.

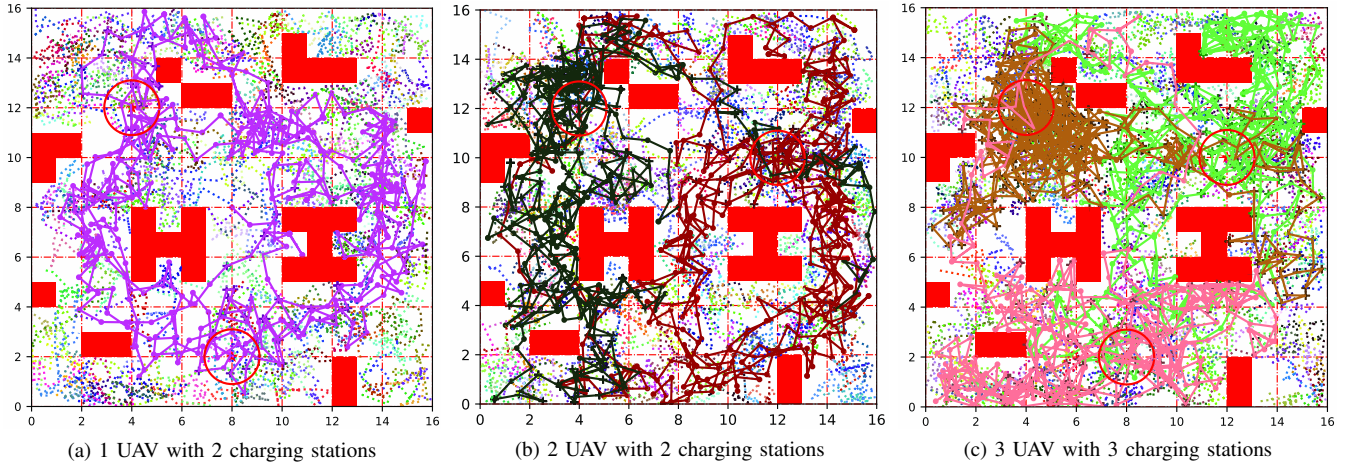


Fig. 3: After executing 500 time steps, the flight traces of the UAVs are depicted, with lines of different colors indicating individual UAVs. Each charging station is positioned at the center of a red circle, while small dots in various colors illustrate the locations of devices and their Gaussian movements. In the coordinates, 1 unit = 10m.

### F. Complexity and Convergence Analysis

The proposed  $j$ -PPO+EN-ConvNTM algorithm is built upon the foundation of PPO, with a particular focus on its convergence behavior and computational complexity in practical application scenarios. Theoretical studies have demonstrated that, under certain conditions, PPO can converge to a neighborhood of the global optimum with high probability. Specifically, if the reward function is smooth and the policy space is compact, PPO achieves a sublinear convergence rate, typically characterized as follows [34]

$$\mathbb{E}[|\theta_k - \theta^*|^2] \leq \frac{F}{k}, \quad (42)$$

where  $\theta^*$  denotes the optimal policy parameter,  $F$  is a constant depending on the dimensionality of the problem and the smoothness of the functions involved, and  $k$  is the number of policy updates. This relation reflects how the policy parameters gradually approach the optimum as the number of iterations increases.

Additionally, In the context of our proposed multi-UAV path planning problem, the high dimensionality of both the state and action spaces imposes stricter requirements on the convergence rate and stability of the algorithm. Given the hybrid nature of continuous and discrete action decisions in the  $j$ -PPO algorithm, particularly the formulation of the hybrid probability ratio  $g_n^{\text{hybrid}}(\theta)$ , we place special emphasis on the selection of the clipping parameter  $\varepsilon$  during implementation. This ensures that each policy update remains within a moderate range, thereby promoting the stable convergence.

In the case of linear MDP, if the policy update magnitude is properly controlled (i.e., through a suitable choice of the clipping parameter  $\varepsilon$ ), the complexity of PPO can be typically bounded by a polynomial function as [34]

$$O\left(\frac{d_{sa} a^2 N_h^3}{\varepsilon^2}\right), \quad (43)$$

where  $d_{sa}$  represents the effective dimensionality of the state-action space,  $N_h$  is the episode horizon.

## V. PERFORMANCE EVALUATION

### A. Setup and Evaluation Metrics

The implementation of our proposed model  $j$ -PPO+EN-ConvNTM is conducted using PyTorch 2.1.0. All experiments are performed on an Ubuntu 20.04 server equipped with two NVIDIA™ GeForce RTX® 4090 graphics cards. We organize four sets of experiments to comprehensively evaluate the proposed model: the first focuses on the impact of critical hyperparameters, the subsequent two involve performance comparisons against four baseline models, namely  $j$ -PPO,  $j$ -PPO+ConvNTM,  $j$ -PPO+ConvLSTM, and  $j$ -PPO+NeuralMap, and the final one conducts extensive comparisons with existing state-of-the-art DRL algorithms. To fulfill the objectives outlined in Section III, we utilize four metrics: the data collection coefficient  $\psi_n$ , spatial equity index  $f_n$ , energy expenditure coefficient  $\kappa_n$ , and the equilibrium efficiency metric  $\Omega_n$ . Notably,  $\Omega_n$  serves as a comprehensive evaluation criterion for selecting hyperparameters and determining the superior solution, and the multiplication-division-base cost function  $\Omega_n = \psi_n f_n / \kappa_n$  is employed.

In the experiments, 256 mobile IoT devices are deployed within a 160m × 160m area to simulate high user density scenarios. Additionally, penalties are introduced when the distance between two UAVs is too small, when a UAV is too close to an obstacle, or when the UAV's battery is about to be depleted. As detailed in Algorithm 1, we conduct training over 3000 iterations with an update horizon  $\tilde{N} = 8$  (resulting in 3000 models), selecting the most equilibrium efficiency model for further testing. During the training phase, we evaluate every model with 500 time steps per episodes, and results from the final time step are adopted as the reward. The hyperparameters are directly follow the standard PPO implementation as reported in [35] with  $K = 5$ ,  $\gamma = 0.99$ ,  $l_r = 0.00025$ ,  $M = 400$ . The remaining parameters are set as follows: The UAV flight altitude  $h_u = 35m$ , flight speed  $v_u = 10m/s$ , weighting factors  $\eta_u = 0.1$  and  $\eta_d = 1$ , and the charging ratio  $q = 0.25$ .

TABLE I: The influence of loss coefficient  $c_1$ , entropy coefficient  $c_2$ , and weight factor  $c_3$ .

|          | Loss coefficient ( $c_1$ ) |        |               |        |        |               |        |        | Entropy coefficient ( $c_2$ ) |               |        |               |        |               |        |               | Weight factor ( $c_3$ ) |        |  |  |  |
|----------|----------------------------|--------|---------------|--------|--------|---------------|--------|--------|-------------------------------|---------------|--------|---------------|--------|---------------|--------|---------------|-------------------------|--------|--|--|--|
|          | 0.001                      | 0.01   | 0.1           | 0.3    | 0.5    | 0.7           | 0.9    | 0      | 0.001                         | 0.01          | 0.1    | 0.3           | 0.5    | 0.1           | 0.3    | 0.5           | 0.7                     | 0.9    |  |  |  |
| $\Omega$ | 0.8133                     | 0.6619 | <b>0.9849</b> | 0.7371 | 0.8725 | 0.6313        | 0.7032 | 0.6178 | 0.8312                        | <b>0.9891</b> | 0.6489 | 0.7561        | 0.7904 | 0.7683        | 0.7847 | <b>0.9827</b> | 0.6148                  | 0.8472 |  |  |  |
| $\kappa$ | 0.7316                     | 0.7282 | 0.7329        | 0.7245 | 0.7294 | <b>0.7330</b> | 0.7314 | 0.7276 | 0.7262                        | 0.7298        | 0.7216 | <b>0.7299</b> | 0.7259 | <b>0.7351</b> | 0.7215 | 0.7347        | 0.7201                  | 0.7223 |  |  |  |
| $\psi$   | 0.8973                     | 0.9103 | <b>0.9449</b> | 0.9054 | 0.8533 | 0.8916        | 0.8917 | 0.8776 | 0.8131                        | <b>0.9449</b> | 0.8464 | 0.8270        | 0.8059 | 0.8856        | 0.8981 | <b>0.9449</b> | 0.8742                  | 0.8328 |  |  |  |
| $f$      | 0.6631                     | 0.5295 | <b>0.7639</b> | 0.5898 | 0.7458 | 0.5190        | 0.5768 | 0.5122 | 0.7424                        | <b>0.7639</b> | 0.5532 | 0.6673        | 0.7119 | 0.6377        | 0.6304 | <b>0.7641</b> | 0.5064                  | 0.7348 |  |  |  |

### B. Illustrative Flight Trajectory

Matplotlib 3.8.4 is employed to develop a realistic UAV simulation environment, into which all model data are integrated. Observations reveal that the simulations demonstrate that UAVs effectively learn to navigate to designated charging stations for recharging, followed by resuming their MEC communication missions. As illustrated in Fig. 3 (a, b, c), the flight patterns of UAVs evolve dynamically with an increase in their numbers. This adaptation is attributed to our proposed approach, which successfully assigns each UAV a distinct and minimally overlapping operational zone. This strategy enhances data collection efficiency while minimizing energy consumption associated with flight. Additionally, Fig. 3 (b, c) highlights that increasing the number of charging stations significantly influences the overall flight patterns. UAVs tend to select closer charging stations to further reduce travel energy costs.

### C. Impact of Hyperparameters

For the EN-ConvNTM and  $j$ -PPO models, we meticulously select hyperparameters  $c_1$ ,  $c_2$ , and  $c_3$  while maintaining the numbers of UAVs and charging stations at  $U=2$  and  $Z=1$ , respectively. The specific configurations are as follows: when tuning  $c_1$ , we set  $c_2=0.01$  and  $c_3=0.5$ ; when tuning  $c_2$ , we set  $c_1=0.1$  and  $c_3=0.5$ ; when tuning  $c_3$ , we set  $c_1=0.1$  and  $c_2=0.01$ . The corresponding results are presented in Tables I.

For the  $j$ -PPO model, the loss coefficient  $c_1$  and the entropy coefficient  $c_2$  play crucial roles in the loss function, as detailed in Eqn. (40a). If either  $c_1$  or  $c_2$  is set relatively high, these terms will dominate the loss function, potentially overshadowing other essential components. Conversely, if these coefficients are set too low, their corresponding terms in the loss function may become negligible, further degrading the model's performance. We have determined the optimal settings for these parameters to be  $c_1 = 0.1$  and  $c_2 = 0.01$ . Similarly, an inappropriate value for the weight factor  $c_3$  can lead to an imbalance in the hybrid probability ratio described in Eqn. (40c), where either the continuous or discrete action probability ratio becomes overly dominant. This imbalance can result in unstable policy behaviors and reduced energy efficiency. In our experiments, the optimal setting for  $c_3$  is determined to be 0.5, which effectively can balance the contributions of different action probability ratios.

### D. Comparisons with Other Baselines

Our findings are presented as performance comparisons in Fig. 4, Fig. 5, and Fig. 6. The number of charging stations is set to  $Z=2$  in Fig. 4 and Fig. 6 (a), and the number of

UAVs is kept at  $U=2$  in Fig. 5 and Fig. 6 (b). It should be noted that different initial conditions in simulations typically lead to performance fluctuations. To better represent the performance comparison, we use boxplots (also known as box-and-whisker plots) to illustrate the distribution of algorithm performance across multiple simulation runs. Specifically, the boxplot includes comprehensive statistical data, such as the minimum, first quartile, median, third quartile, and maximum, to provide a holistic overview of the algorithm's performance. Additionally, since adjacent boxplots may overlap with each other, causing ambiguity in the display, we have slightly adjusted the horizontal coordinates corresponding to each algorithm (the actual horizontal coordinates remain as 1, 2, 3, 4, and 5) to avoid overlap and better display the performance curves.

From Fig. 4, Fig. 5, and Fig. 6, the following conclusions can be derived:

1) Equilibrium Efficiency Comparisons: The proposed  $j$ -PPO+EN-ConvNTM outperforms all baseline models in terms of equilibrium efficiency. In our experimental setup, UAVs actively navigate within the target area, thus necessitating a robust spatiotemporal module to fully extract geographical and trajectory information from the data for optimal decision-making. The standard  $j$ -PPO algorithm lacks mechanisms to effectively consider the locations of charging stations, which limits its ability to strategically balance charging and data collection, thereby leading to suboptimal performance. The  $j$ -PPO+NeuralMap performs inadequately because it simply stores a 1-D vector in a 2-D map based on a real-world 2-D position. However, in our scenario, where a single agent controls multiple UAVs, the  $j$ -PPO+NeuralMap results in identical vectors being stored for different UAVs. For example, when the number of UAVs is two, the equilibrium efficiency of our proposed algorithm is 76.2% higher than that of  $j$ -PPO+NeuralMap. This likely arises from the potential loss of critical spatial information by NeuralMap during the compression of 3-D observations into 1-D vectors. Additionally,  $j$ -PPO+ConvLSTM relies solely on current observations without integrating historical data, which is crucial for informed decision-making, leading to performance degradation.

Both  $j$ -PPO+EN-ConvNTM and  $j$ -PPO+ConvNTM address this issue by effectively modeling spatial and temporal correlations. However, with the incorporation of STN and attention mechanism, EN-ConvNTM handles longer sequences more efficiently than ConvNTM, making  $j$ -PPO+EN-ConvNTM consistently more effective. For instance, as illustrated in Fig. 4, as the number of UAVs increases from 1 to 5,  $j$ -PPO+EN-ConvNTM achieves energy efficiency improvements by 9.91%, 8.06%, 17.60%, 14.95%, and 21.21%, compared

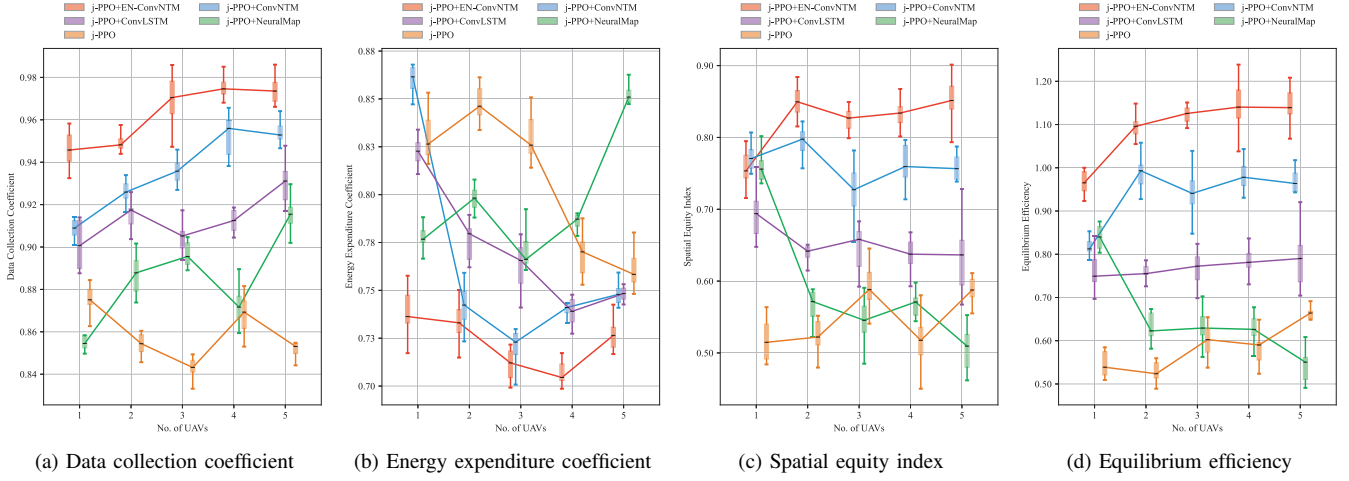


Fig. 4: The results of increasing the number of UAVs from 1 to 5, while maintaining the presence of 2 charging stations.

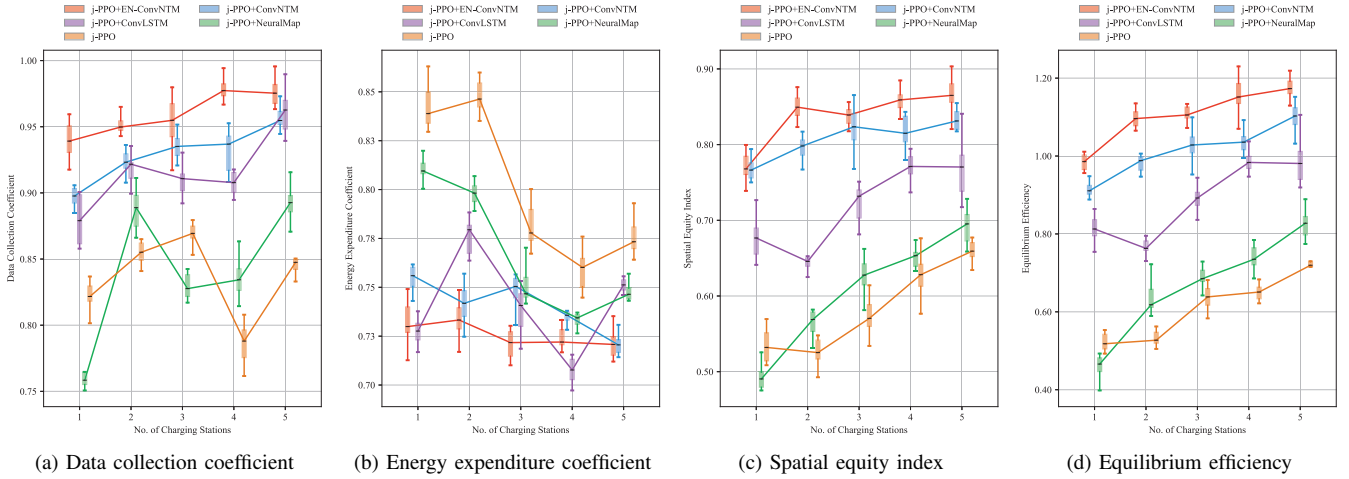


Fig. 5: The results of increasing the number of charging stations from 1 to 5, while maintaining the presence of 2 UAVs.

to *j*-PPO+ConvNTM, respectively. Similarly, as depicted in Fig. 5, as the number of charging stations increases from 1 to 5, *j*-PPO+EN-ConvNTM respectively boosts equilibrium efficiency by 7.34%, 11.30%, 7.62%, 11.65%, and 6.14% compared to *j*-PPO+ConvNTM.

2) Impact of UAV Numbers: As depicted in Fig. 4, the number of UAVs significantly affects four key metrics. *j*-PPO, *j*-PPO+ConvNTM, *j*-PPO+ConvLSTM, *j*-PPO+NeuralMap, and *j*-PPO+EN-ConvNTM exhibit an convergence trend in equilibrium efficiency, fairness, and data collection ratio, which eventually stabilizes with minor fluctuations. In contrast, the energy expenditure coefficient generally follows an inverted U-shaped trend, initially decreasing and then increasing as the number of UAVs increases. This trend indicates that an optimal number of UAVs can effectively assign the data collection tasks more fairly, thereby reducing overall energy consumption. However, with a fixed number of charging stations, an excessive number of UAVs may increase energy consumption. This is because UAVs may need to flight longer distances to reach the charging stations or to collect data from remote areas, potentially degrade the energy efficiency.

3) Impact of Charging Station Numbers: Fig. 5 investigates how variations in charging station numbers affect the same four metrics. As the number of charging stations increases, improvements are generally observed in equilibrium efficiency, fairness, and data collection ratio. For the energy expenditure coefficient, a reduction is typically derived. These findings indicate that an increase in the number of charging stations has a positive impact on overall performance. This can be attributed to reduced flight distances required for UAVs to recharge. Such adjustments allow UAVs to operate within closer to their activity areas, which is also reflected by the benefit in equilibrium efficiency.

4) Comparisons with Existing DRL Algorithms: Fig. 6 presents a comprehensive performance comparison between the proposed *j*-PPO+EN-ConvNTM algorithm and state-of-the-art DRL algorithms, including DDPG, A2C, TD3, and DQN. Through rigorous empirical analysis, the proposed intelligent PPO-based scheme demonstrates distinctive superiority in equilibrium efficiency compared to existing DRL-based approaches. Notably, the proposed algorithm consistently achieves the highest equilibrium efficiency, followed



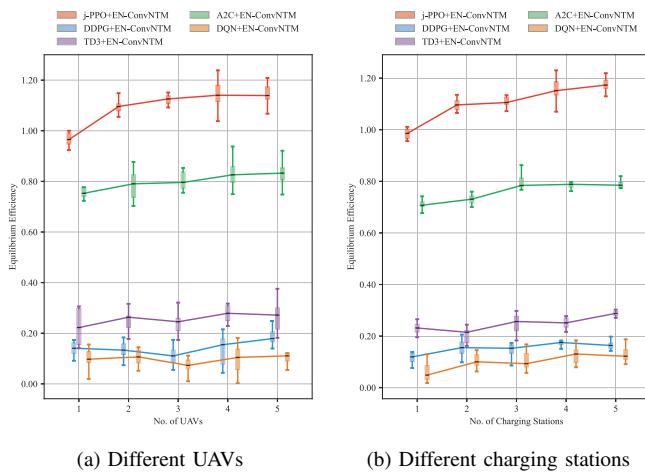


Fig. 6: Performance comparisons with existing DRL algorithms.

by the A2C-based algorithm, across varying numbers of UAVs and charging stations. In contrast, DDPG, TD3, and DQN exhibit substantial convergence challenges and severe performance degradation. This divergence can be attributed to the inherent limitations of most existing DRL algorithms in addressing joint continuous-discrete action decision-making under high-density user mobility scenarios. Specifically, such environments require simultaneous optimization of continuous control variables (e.g., UAV trajectory planning) and discrete decision variables (e.g., offloading strategy and charging indicator), which exceeds the capability of algorithms relying on continuous or discrete action space frameworks.

## VI. CONCLUSION

In this study, we tackle the challenge of path planning for multiple UAVs in high-density mobile device environments with multiple charging stations available. The optimization problem is structured using a POMDP, and an innovative deep reinforcement learning framework named  $j$ -PPO+EN-ConvNTM is proposed. This framework integrates  $j$ -PPO, a modified version of the PPO algorithm supporting both continuous and discrete action controls. Additionally, it incorporates a novel sequential module, EN-ConvNTM, which leverages a STN to process and utilize extensive spatiotemporal sequences. Comprehensive experiments are conducted to evaluate UAV flight trajectory design, hyperparameter settings, and performance comparisons with four baseline models and existing DRL algorithms. The results strongly demonstrate the superior efficacy and performance improvements of our proposed solution. It is essential to recognize that the proposed  $j$ -PPO+EN-ConvNTM algorithm has undergone numerical simulations to demonstrate its effectiveness. Future work should involve testing the  $j$ -PPO+EN-ConvNTM algorithm on a real-world UAV-assisted MEC system to validate its practical performance, and other challenges, such as hardware limitations and uplink interferences, require further investigation.

## REFERENCES

- [1] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on multi-access edge computing for internet of things realiza-

- tion," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 2961–2991, 2018.
- [2] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for iot applications: A learning-based approach," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 5, pp. 1117–1129, 2019.
- [3] M. Mozaffari, W. Saad, M. Bennis, and M. Debbah, "Mobile unmanned aerial vehicles (uavs) for energy-efficient internet of things communications," *IEEE Transactions on Wireless Communications*, vol. 16, no. 11, pp. 7574–7589, 2017.
- [4] Z. Wei, M. Zhu, N. Zhang, L. Wang, Y. Zou, Z. Meng, H. Wu, and Z. Feng, "Uav-assisted data collection for internet of things: A survey," *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 15460–15483, 2022.
- [5] Y. Xu, T. Zhang, Y. Liu, D. Yang, L. Xiao, and M. Tao, "Uav-assisted mec networks with aerial and ground cooperation," *IEEE Transactions on Wireless Communications*, vol. 20, no. 12, pp. 7712–7727, 2021.
- [6] G. Chen, X. B. Zhai, and C. Li, "Joint optimization of trajectory and user association via reinforcement learning for uav-aided data collection in wireless networks," *IEEE Transactions on Wireless Communications*, vol. 22, no. 5, pp. 3128–3143, 2023.
- [7] L. Wang, Y. Li, Y. Chen, T. Li, and Z. Yin, "Air-ground coordinated mec: Joint task, time allocation and trajectory design," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 3, pp. 4728–4743, 2025.
- [8] Y. Zhang, Z. Kuang, Y. Feng, and F. Hou, "Task offloading and trajectory optimization for secure communications in dynamic user multi-uav mec systems," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 14427–14440, 2024.
- [9] K. Tange, M. De Donno, X. Fafoutis, and N. Dragoni, "A systematic survey of industrial internet of things security: Requirements and fog computing opportunities," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2489–2520, 2020.
- [10] C. H. Liu, C. Piao, and J. Tang, "Energy-efficient uav crowdsensing with multiple charging stations by deep learning," in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*. IEEE, 2020, pp. 199–208.
- [11] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [12] M. Mozaffari, W. Saad, M. Bennis, M. Debbah, and Y.-H. Nam, "A tutorial on uavs for wireless networks: Applications, challenges, and open problems," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2334–2360, 2019.
- [13] Y. Hu, M. Patel, D. Sabella, N. Sprecher, and V. Young, "Mobile edge computing—a key technology towards 5g," *ETSI White Paper*, vol. 11, no. 11, 2015.
- [14] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: Opportunities and challenges," *IEEE Communications Magazine*, vol. 54, no. 5, pp. 36–42, 2016.
- [15] Q. Zhen, H. Shoushuai, W. Hai, Q. Yuben, D. Haipeng, X. Fei, W. Zhenhua, and L. Hailong, "Air-ground collaborative mobile edge computing: Architecture, challenges, and opportunities," *China Communications*, vol. 21, no. 5, pp. 1–16, 2024.
- [16] M. Darchini-Tabrizi, A. Pakdaman-Donyavi, R. Entezari-Maleki, and L. Sousa, "Performance enhancement of uav-enabled mec systems through intelligent task offloading and resource allocation," *Computer Networks*, vol. 264, p. 111280, 2025.
- [17] M. H. Adnan, Z. A. Zukarnain, and O. A. Amodu, "Fundamental design aspects of uav-enabled mec systems: A review on models, challenges, and future opportunities," *Computer Science Review*, vol. 51, p. 100615, 2024.
- [18] S. A. Ullah, S. A. Hassan, H. Abou-Zeid, H. K. Qureshi, H. Jung, A. Mahmood, M. Gidlund, M. A. Imran, and E. Hossain, "Convergence of mec and drl in non-terrestrial wireless networks: Key innovations, challenges, and future pathways," *IEEE Communications Surveys & Tutorials*, 2025.
- [19] Y. Gao, X. Yuan, D. Yang, Y. Hu, Y. Cao, and A. Schmeink, "Uav-assisted mec system with mobile ground terminals: Drl-based joint terminal scheduling and uav 3d trajectory design," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 7, pp. 10164–10180, 2024.
- [20] L. Wang, B. Shen, L. Ma, Y. Zhang, Y. Zhao, H. Guo, Z. Yu, and B. Guo, "Joint task offloading and migration optimization in uav-enabled dynamic mec networks," *IEEE Transactions on Services Computing*, 2025.

- [21] Z. Wang, H. Wang, L. Liu, E. Sun, H. Zhang, Z. Li, C. Fang, and M. Li, "Dynamic trajectory design for multi-uav-assisted mobile edge computing," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 3, pp. 4684–4697, 2025.
- [22] J. Zhang, L. Zhou, F. Zhou, B.-C. Seet, H. Zhang, Z. Cai, and J. Wei, "Computation-efficient offloading and trajectory scheduling for multi-uav assisted mobile edge computing," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 2, pp. 2114–2125, 2020.
- [23] M. Chen, U. Challita, W. Saad, C. Yin, and M. Debbah, "Artificial neural networks-based machine learning for wireless networks: A tutorial," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3039–3071, 2019.
- [24] P. McEnroe, S. Wang, and M. Liyanage, "A survey on the convergence of edge computing and ai for uavs: Opportunities and challenges," *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 15 435–15 459, 2022.
- [25] Z. Ning, H. Hu, X. Wang, L. Guo, S. Guo, G. Wang, and X. Gao, "Mobile edge computing and machine learning in the internet of unmanned aerial vehicles: A survey," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–31, 2023.
- [26] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in mec-and uav-assisted vehicular networks," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 131–141, 2020.
- [27] A. A. Al-Bakhrani, M. Li, M. S. Obaidat, and G. A. Amran, "Moalf-uav-mec: Adaptive multi-objective optimization for uav-assisted mobile edge computing in dynamic iot environments," *IEEE Internet of Things Journal*, vol. 12, no. 12, pp. 20 736–20 756, 2025.
- [28] M. Hevesli, A. M. Seid, A. Erbad, and M. Abdallah, "Task offloading optimization in digital twin assisted mec-enabled air-ground iiot 6g networks," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 11, pp. 17 527–17 542, 2024.
- [29] Y. Chen, H. Kang, J. Li, G. Sun, B. Wang, J. Wang, C. Liang, S. Liang, and D. Niyato, "Joint resource management for energy-efficient uav-assisted swipt-mec: A deep reinforcement learning approach," *IEEE Internet of Things Journal*, 2025.
- [30] A. Graves, G. Wayne, M. Reynolds *et al.*, "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, pp. 471–476, 2016.
- [31] Q. Liu, L. Shi, L. Sun, J. Li, M. Ding, and F. Shu, "Path planning for uav-mounted mobile edge computing with deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 5723–5728, 2020.
- [32] G. Zheng, C. Xu, M. Wen, and X. Zhao, "Service caching based aerial cooperative computing and resource allocation in multi-uav enabled mec systems," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 10, pp. 10 934–10 947, 2022.
- [33] M. Asim, M. ELAffendi, and A. A. Abd El-Latif, "Multi-irs and multi-uav-assisted mec system for 5g/6g networks: Efficient joint trajectory optimization and passive beamforming framework," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 4553–4564, 2023.
- [34] H. Zhong and T. Zhang, "A theoretical analysis of optimistic proximal policy optimization in linear markov decision processes," *Advances in Neural Information Processing Systems*, vol. 36, pp. 73 666–73 690, 2023.
- [35] I. Kostrikov, "Pytorch implementations of asynchronous advantage actor critic (a3c), proximal policy optimization (ppo) and more," <https://github.com/ikostrikov/pytorch-a2c-ppo-acktr-gail>, 2019.